

SpecTrace

[English version](#)

SpecTrace извлекает нормативные требования из технической спецификации и привязывает каждое из них к точному фрагменту исходного текста. Затем система предлагает тест-кейсы, прослеживаемые до этих требований, и показывает, у каких требований тест-кейсов нет. Каждое утверждение в её выводе можно проверить кодом по самому документу.

В этом репозитории — концепция, согласованные требования, решения и записи о приёме. Код, данные и все результаты — в [spectrace-dev](#).

Проблема и для кого

- **Пользователь.** Тестировщику или разработчику дали спецификацию. По ней нужно составить тест-кейсы и показать, что ничто в ней не осталось непроверенным.
- **Вручную** эта работа точна, но медленна.
- **С чат-ассистентом** она быстра, но названное им требование можно проверить, только перечитав весь документ, — а именно эту работу ассистент и должен был снять.
- **Что меняет SpecTrace:** каждое требование привязано к точному месту в источнике. Всё, что модель утверждает, но чего нет в тексте, откладывается и показывается отдельно. Каждое требование без тест-кейса видно как названный пробел.

Почему такой подход

Модель предлагает, код проверяет. Семь правил соблюдаются при любой модели:

- Модель возвращает только текст цитаты. Код находит, где этот текст стоит в документе, — точным совпадением после схлопывания пробелов — и никогда не принимает позицию от модели (P1, P2).
- Цитата, которую не удалось найти, отбрасывается и показывается отдельно. Цитата, найденная больше одного раза, уходит человеку (P3, P6).
- Каждый вызов модели кэшируется по полному запросу, поэтому весь прогон воспроизводится офлайн, без ключа (P5).
- Тест-кейс не может существовать без требования. Вывод никогда не утверждает, что спецификация покрыта полностью, — только что в реестре нет непокрытых требований (P7).

Другая модель меняет числа, но не эти гарантии. Главные выборы, на которых они держатся, с альтернативами и доказательствами — в разделе [Ключевые решения](#).

Что сделано

Веха	Результат	Статус
M1	На входе один документ, на выходе — проверенный реестр, прослеживаемые тест-кейсы и матрица с видимыми пробелами. Прогон воспроизводится офлайн в CI без ключа.	Принята 26 сентября 2026 г., тег <code>m1-accepted</code> в <code>spectrace-dev</code> и <code>SpecTrace-docs</code>
M2	Рецензент принимает, правит или отклоняет каждый кейс, и каждое решение сохраняется. Измеренное сравнение с подходами без проверки на двух документах. Интерфейс рецензирования по публичному адресу.	Принята 1 октября 2026 г., тег <code>m2-accepted</code> в <code>spectrace-dev</code> и <code>SpecTrace-docs</code>
M3	Эти документы, решение о живом сервисе и демонстрация на 5–7 минут	Завершена 3 октября 2026 г., тег <code>defense</code> в <code>spectrace-dev</code> и <code>SpecTrace-docs</code>

- **Система:** конвейер и командная строка на .NET и интерфейс рецензирования с серверным рендерингом. Она работает публично по адресу `spectrace-5zrm6uxcja-lz.a.run.app`: документы корпуса воспроизводятся из закоммиченного кэша, а новый документ обрабатывается вживую.
- **Кто что делал:** большую часть кода и документов написал агент-программист. Студент задал рамки, принял решения, записанные как его, разметил эталон, записал ответы чата и рецензирует эталонный прогон. Запись — в `docs/agent-worklog.md`.

Что подтверждает результат

Каждая цифра — в указанном файле, который любой может пересоздать одной командой. Объём выборки указан рядом.

- **Что задерживает проверка.** На RFC 6902, с той же моделью, температурой 0 и тем же верификатором:
 - 81,3 % цитат наивного промпта нет в документе дословно (13 из 16). Почти все они изменены одним способом: двойные кавычки заменены одинарными.
 - Цитаты конвейера: 0,0 % (0 из 18).
 - Ответ чат-ассистента, записанный вручную: все 18 его цитат нашлись.
 - На RFC 10050, опубликованном после даты среза знаний модели, три ветки близки: чат 9,5 %, наивный промпт 9,1 %, конвейер 3,3 %.

Один прогон на ветку, от 16 до 30 утверждений в каждом: `experiments/headline.md`, `experiments/error-analysis.md` §2.

- **Качество против эталона, размеченного вручную.** У реестра конвейера точность 100 % (12 из 12), полнота 63,2 % (12 из 19), F1 77,4 %. До того как проверка задержала часть цитат, полнота была 94,7 %. 6 задержанных требований ждут в очереди решений человека: 4 — цитаты, которые встречаются в документе дважды, и 2 — из одного предложения, которое модель прочитала двумя способами. 19 эталонных требований, один разметчик: `experiments/headline.md`, `experiments/error-analysis.md` §3.

- **Как часто рецензент оставлял предложения.** Так проект измеряет, помогают ли его тест-кейсы: сколько из них рецензент принимает как есть, правит или отклоняет. На эталонном прогоне из 23 кейсов 11 приняты как есть, 9 исправлены и 3 отклонены. Рецензент один, он же автор, документ один. 21 из 23 решений принято, когда перед рецензентом было мнение агента о каждом кейсе, по его собственной просьбе: [experiments/review/](#) , [experiments/error-analysis.md](#) §9.
- **Любой может это повторить.** Одна команда воспроизводит эталонный прогон офлайн, без ключа.
 - CI выполняет эту команду на Linux и Windows при каждом push и сравнивает каждый файл побайтно, кроме времени запуска и хеша коммита.
 - CI также пересоздаёт главную метрику и показатели качества и падает, если хоть одна цифра расходится с закоммиченными файлами.
 - Все 40 прогонов CI с 20 сентября по 1 октября 2026 г. прошли успешно: [CI](#).

Почему результат считается верным и готовым

M1 и M2 приняты после проверки, критерий за критерием, по доказательствам, перепроверенным на принятом коммите.

Верна — в том смысле, который проект задаёт себе сам.

- Каждое требование в реестре — дословная цитата, которую код нашёл в источнике.
- Каждый тест-кейс ссылается на требование.
- Каждое требование без тест-кейса остаётся видимым пробелом, пока человек не решит иначе.

Эти свойства обеспечены устройством системы и проверяются тестами и инвариантами I1–I8 при каждом push. Все 44 прогона CI с 20 сентября по 2 октября 2026 г. прошли успешно. Главная метрика и показатели качества не заявлены, а пересчитываются: одна команда пересоздаёт их из закоммиченного кэша, и CI падает, если хоть одна цифра изменится. Там, где система не могла решить сама, — цитата найдена дважды, предложение прочитано двумя способами, к требованию не удалось написать тест-кейс, — она передала вопрос человеку, а не стала угадывать.

Готова к тому, что требует курс, и не более. Человек вне проекта может повторить прогон, прочитать, почему принято каждое решение, и увидеть, как система обрабатывает документ, который никто заранее не записывал. Но:

- её качество измерено на одном документе, с одной моделью и одним разметчиком;
- рецензирование провёл один человек — сам автор.

Эти границы названы в [docs/limitations.md](#) , а не скрыты.

Где проходят границы

- `docs/limitations.md` — что эти результаты позволяют проекту утверждать, а что нет.
- `docs/privacy-safety.md` — что прогон отправляет модели и куда. Там же — ограничение использования, вопреки которому работает публичный сервис.
- NFR-04 не выполнено: провайдер и модель — константы в `LlmClientFactory.cs`, поэтому другой провайдер или модель — это правка кода, а не настройки.

Как повторить

Нужны Git и .NET SDK 10.0, ключ не нужен, сеть — только для клонирования и восстановления пакетов:

```
git clone --branch defense https://github.com/KaunazDagaz/SpecTrace-dev.git
cd SpecTrace-dev
dotnet run --project src/SpecTrace.Cli -- run --document corpus/rfc6902.txt --offline --out runs/reference
git status
```

`git status` показывает только `runs/reference/manifest.json`: в нём меняются время запуска и хеш коммита. Остальные команды, интерфейс рецензирования и эксперимент — в [README репозитория spectrace-dev](#).

Ключевые решения

Решение	Альтернативы	Доказательства	Статус
.NET и C#	Python, Node и TypeScript	Навыки студента и код, который можно переиспользовать в дипломе	Сохранено
Модель возвращает текст цитаты, позицию находит код	Позиции, которые сообщает модель	Позицию цитаты код проверяет точно, смещения от модели проверить нельзя. Точным совпадением эталонный прогон нашёл 14 из 18 цитат по одному разу, а 4, найденные дважды, отправил человеку.	Сохранено
Каждый вызов модели кэшируется по полному запросу, температура 0	Без кэша, живой вызов при каждом прогоне	CI воспроизводит эталонный прогон офлайн на Linux и Windows и сравнивает его побайтно	Сохранено
Модель — Gemini 3.5 Flash-Lite, с 23 сентября	Gemini 3.5 Flash, как планировалось	Бесплатный уровень Flash давал 20 запросов в день — меньше одного прогона, Flash-Lite давал 500	Запланированная модель заменена
Без разбиения на фрагменты: весь документ одним запросом	Извлечение по фрагментам	Заранее зафиксированное правило не может сработать на RFC 6902: в его последней трети нет ни одного эталонного требования. Это отсутствие доказательств, а не доказательство (<code>chunking-decision.md</code>).	Сохранено

Решение	Альтернативы	Доказательства	Статус
Размер эталона задают правила, зафиксированные заранее: 19 требований	30–50 требований, как планировалось сначала	Поиск по ключевым словам находит 20 предложений-кандидатов; по зафиксированным правилам оставлены 18, из них получилось 19 требований. Чтобы получить 30, пришлось бы считать то, что извлечение исключает, или добирать до числа.	Изменено; ждёт одобрения руководителя
Базовый промпт просит цитаты	Первый базовый промпт, просивший только тест-кейсы	В ответе из одних тест-кейсов нечего сверять с источником	Первый промпт заменён
Журнал рецензирования вне папки прогона	Журнал внутри <code>runs/{runId}/</code>	Журнал внутри эталонного прогона сломал бы его побайтный тест, а случайное решение в закоммиченной папке уже нельзя было бы удалить	Изменено в SPEC-13
Интерфейс рецензирования может запускать прогон	Прогоны только из командной строки	Для рецензирования и демонстрации нужен человек, который рецензирует настоящий прогон в браузере	Сохранено